

AI Agents : Tools & Building an Agent

From LLMs to Tools to Agents

Before understanding AI agents, we first need to understand one fundamental limitation of an LLM.

1. What Is an LLM Actually Built to Do?

At its core, an LLM performs one fundamental task:

Given some input tokens, predict the next token.

Conceptually:

```
Input Tokens
  ↓
Transformer
  ↓
Probability Distribution
over Vocabulary
  ↓
Next Token
  ↓
Next Token
  ↓
Next Token
```

This mechanism can generate surprisingly intelligent responses, but there is an important distinction between **generating an action** and **performing an action**.

Suppose we ask:

Create a folder called `portfolio`.

An LLM can generate:

`mkdir portfolio`

But generating the text `mkdir portfolio` does not mean the command was actually executed.

The LLM generated characters representing an action.

It did not perform the action.

2. LLM = Brain, Tools = Hands

A useful mental model is to imagine an intelligent person locked inside a room.

They may have:

- enormous knowledge
- reasoning ability
- language ability

But they have no hands.

If you ask:

| Turn on the AC.

They understand exactly what you want and may tell you:

| Press the power button on the remote.

But they cannot physically press the button themselves.

A raw LLM has a similar limitation.

```
TEXT IN
  ↓
  LLM
  ↓
TEXT OUT
```

By itself, an LLM does not automatically have access to things such as:

- databases
- file systems
- live internet data
- calculators
- terminals
- email systems
- calendars
- payment systems
- weather services

These capabilities have to be provided by the surrounding application.

A simple mental model is:

```
LLM = Brain
Tools = Hands
```

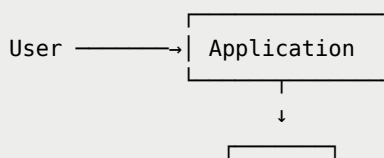
Therefore:

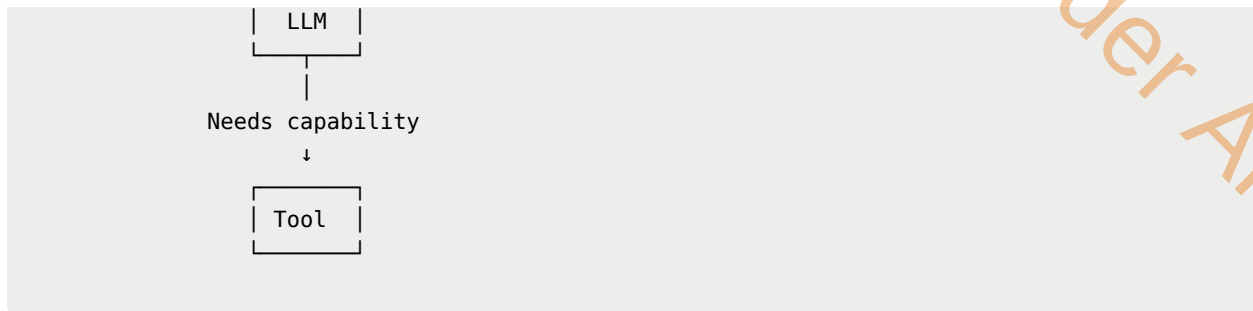
```
LLM + Tools
  ↓
System capable of taking actions
```

3. How Applications Give LLMs Capabilities

Applications such as modern AI assistants are more than just an LLM.

Conceptually:





The application surrounding the LLM gives it access to external capabilities.
This is where **tool calling** begins.

4. Why Give an LLM a Calculator?

Consider this question:

| What is 938475×736294 ?

A modern LLM may sometimes calculate the answer correctly, but reliable arithmetic is better handled by deterministic software.

The responsibilities can be separated.

LLM's Job

Understand the intent:

User wants multiplication.

a = 938475
b = 736294

Calculator's Job

Perform the actual computation:

$938475 * 736294$

LLM's Job Again

Convert the result into a natural-language response:

```
The result of multiplying 938,475 by 736,294 is ...
```

We combine two different strengths:

```
LLM
Great at understanding language

+

Program
Great at deterministic computation
```

This pattern becomes extremely powerful when applied beyond calculators.

5. Why Natural Language Is Difficult for Traditional Programs

Suppose we have this Java function:

```
public double add(double a, double b) {
    return a + b;
}
```

A computer understands this perfectly:

```
add(10, 20);
```

But a human may say:

| Add 10 and 20.

or:

| What's ten plus twenty?

or:

| Calculate the sum of 10 and 20.

or even:

| I have 10 apples and I got another 20. How many apples do I have?

For humans, all these statements communicate almost the same intention.

For traditional software, they are completely different strings.

6. Traditional Software Prefers Structure

Without an LLM, we may try something like:

```
if (message.contains("add")) {  
    ...  
}
```

But someone might say:

```
sum
```

So we add another condition.

```
if (message.contains("add") || message.contains("sum")) {  
    ...  
}
```

Then users may say:

```
plus  
combine  
total
```

Or:

| I have ₹300 and received another ₹200.

There may not even be an explicit word such as `add`.

Trying to manually support every possible human expression quickly becomes impractical.

Computers work much better with structured contracts.

For example:

```
{
  "operation": "ADD",
  "a": 10,
  "b": 20
}
```

There is much less ambiguity.

7. One of the Most Important Capabilities of LLMs

LLMs are commonly associated with generating text.

But when building applications, another extremely important capability is:

LLMs can translate messy human intent into structured machine-readable information.

For example:

| Can you divide 450 by 9 for me?

The model can interpret this as:

```
{
  "operation": "DIVIDE",
  "a": 450,
```

```
"b": 9
}
```

The user's language may vary enormously.

Your backend still receives predictable information.

```
Natural Language
  ↓
  LLM
  ↓
Structured Intent
  ↓
Backend Code
```

This creates a bridge between human language and deterministic software.

8. Example: Food Delivery Support

Imagine a customer support chatbot for a food delivery application.

The user says:

| My burger never arrived and I want my money back.

The backend may need something structured like:

```
POST /api/v1/action
```

```
{
  "intent": "REFUND_REQUEST",
  "reason": "ITEM_NOT_DELIVERED",
  "item": "burger"
}
```

Another customer says:

| I ordered a pizza 90 minutes ago. Where is it?

The model may identify:

```
{  
  "intent": "ORDER_STATUS"  
}
```

Instead of manually writing thousands of conditions for every possible sentence:

```
Natural Language  
  ↓  
  LLM  
  ↓  
Structured Intent  
  ↓  
Application Logic
```

The LLM becomes an intelligent translation layer between the user and the software.

9. Why Not Simply Prompt the Model to Return JSON?

We could write:

```
Always answer using JSON.
```

Often, the model will follow the instruction.

But it may occasionally return something like:

```
Sure! Here's the JSON:  
  
{  
  ...  
}
```

Or perhaps:

```
{  
  "operation": "multiplication"  
}
```

while the application expects:

```
MULTIPLY
```

For software integration, we want more than approximately correct JSON.

We want output following a defined contract.

Conceptually:

```
CalculatorRequest {  
  operation = MULTIPLY  
  number1 = 12  
  number2 = 7  
}
```

The idea is:

The LLM can understand the user's intention, but it should express that intention according to a contract understood by our application.

This is where **natural-language intelligence and strongly typed software meet**.

It also represents an interesting combination:

```
Non-deterministic intelligence  
+  
Deterministic software
```

10. What Is a Tool?

Strip away the AI terminology.

A tool is essentially a normal function that the application makes available to the LLM.

Example:

```
public double multiply(double a, double b) {  
    return a * b;  
}
```

There is nothing inherently AI-specific about this method.

It is normal Java code.

The important step is describing this function to the model.

Conceptually, the model receives information similar to:

```
Tool Name:  
multiply  
  
Description:  
Multiplies two numbers.  
  
Parameters:  
a: number  
b: number
```

Now suppose the user asks:

| What is 17 times 42?

The model has information similar to:

```
User request:  
17 times 42  
  
Available tools:  
add  
subtract  
multiply  
divide
```

The model identifies:

```
Required tool = multiply  
  
a = 17  
b = 42
```

Instead of generating the final answer immediately, it can request something conceptually like:

```
{  
  "tool": "multiply",  
  "arguments": {  
    "a": 17,  
    "b": 42  
  }  
}
```

This is called a **tool call** or **function call**.

11. A Tool Definition Is Like an API Contract

Consider a REST API:

```
POST /users
```

It may require:

```
{  
  "name": "...",  
  "email": "..."  
}
```

There is a contract between the client and server.

Tool calling works similarly.

The application tells the model:

```
These are the actions available to you.  
  
This is what each action does.  
  
These are the parameters required.
```

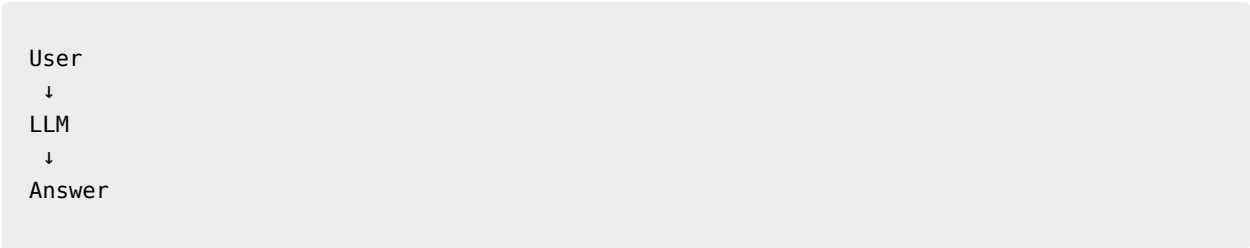
The LLM then decides which action is appropriate.

A useful way to think about it is:

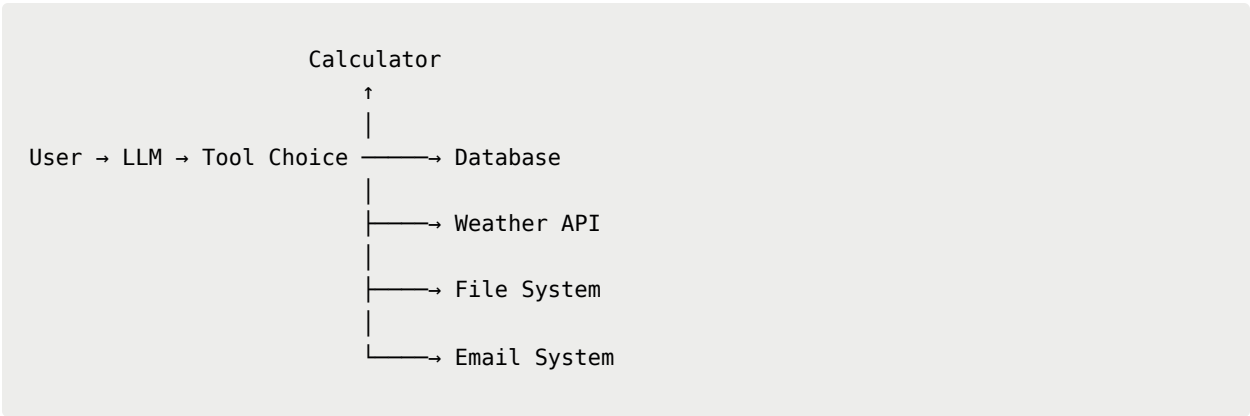
The LLM behaves like an intelligent client that chooses which function or API should be invoked.

12. LLM as a Natural-Language Interface to Software

Without tools:



With tools:



The LLM becomes a natural-language interface over the software ecosystem.

Traditionally:

```
User must adapt to software.
```

Users learn:

- buttons
- forms
- commands
- API formats
- application workflows

With an LLM:

```
Software can understand the user's language.
```

The user describes the goal naturally.

The LLM maps that goal to software actions.

13. Simple Tool-Calling Example

Suppose we have:

```
double multiply(double a, double b) {  
    return a * b;  
}
```

User:

```
I bought 13 boxes containing 24 bottles each.  
How many bottles do I have?
```

The LLM understands:

```
Tool = multiply  
  
a = 13  
b = 24
```

The application executes:

```
multiply(13, 24);
```

Result:

```
312
```

The result goes back to the model.

The model responds:

| You have 312 bottles in total.

This is the basic tool-calling pattern.

14. Practical: Give Our LLM Its First Tool

Until now, our chatbot looked like:

```
USER  
↓  
LLM  
↓  
ANSWER
```

After adding tools:

```
USER  
↓
```

LLM
↓
Does another capability handle this better?
↓
TOOL
↓
RESULT
↓
LLM
↓
ANSWER

We'll begin with a calculator.

15. Calculator Tool with Spring AI

```
@Component
public class CalculatorTools {

    @Tool(description = """
        Performs mathematical calculations.
        Supported operations are:
        add, subtract, multiply, divide, mod and power.
        """)
    public double calculate(
        @ToolParam(description = "Operation to perform")
        String operation,

        @ToolParam(description = "First number")
        double a,

        @ToolParam(description = "Second number")
        double b
    ) {

        return switch (operation.toLowerCase()) {
            case "add" -> a + b;
```

```

case "subtract" -> a - b;
case "multiply" -> a * b;
case "divide" -> a / b;
case "mod" -> a % b;
case "power" -> Math.pow(a, b);

default -> throw new IllegalArgumentException(
    "Unsupported operation"
);
};
}
}

```

The method itself is ordinary Java.

The important parts for the LLM are the tool name, description and parameters.

16. What Does the LLM Actually Receive?

The model does not receive our Java class and inspect implementation details such as:

```
switch (operation)
```

Conceptually, it receives a tool definition closer to:

AVAILABLE TOOL

Name:

calculate

Description:

Performs mathematical calculations.

Inputs:

operation: string

a: number

b: number

When the user asks:

| What is 450 divided by 9?

the model can request:

```
{
  "name": "calculate",
  "arguments": {
    "operation": "divide",
    "a": 450,
    "b": 9
  }
}
```

The surrounding application executes the actual method.

17. Connecting the Tool to ChatClient

```
String response = chatClient
    .prompt()
    .user(message)
    .tools(calculatorTools)
    .call()
    .content();
```

The important point is:

| **The model did not change. The user's prompt did not need to change. We simply gave the model another capability.**

Tools can be supplied for an individual request or configured as default tools on the `ChatClient`.

Example request:

POST /chat

```
{  
  "message": "What is 8745 multiplied by 493?"  
}
```

The LLM can identify that calculation is required and delegate it to the calculator tool.

The internal implementation of that tool could later become much more sophisticated. From the model's perspective, it remains simply another available capability.

18. Tool 2: Current Weather

Calculator and weather demonstrate two slightly different reasons for using tools.

Calculator

The LLM may be able to produce an arithmetic result, but deterministic calculation is more appropriate.

Weather

A raw model does not inherently know reliable live weather conditions.

If the user asks:

| What's the current weather in Delhi?

we need an external source of live information.

So we expose:

```
getCurrentWeather(city)
```

The tool internally calls a weather service.

For example:

```
@Tool(description = """
    Gets the current weather for a city.
    Use this whenever the user asks about current weather,
    temperature, humidity or wind conditions.
    """)
public WeatherResponse getCurrentWeather(
    @ToolParam(description = "Name of the city")
    String city) {

    return weatherService.getCurrentWeather(city);
}
```

The LLM only needs to understand:

```
Tool:
getCurrentWeather

Purpose:
Get current weather

Input:
city
```

It does not need to understand how the weather service works internally.

19. Tool 3: Currency Conversion

Another useful tool is live currency conversion.

```
@Tool(description = """
    Converts money from one currency to another
    using current exchange rates.

    Use ISO currency codes such as USD, INR, EUR and GBP.
    """)
```

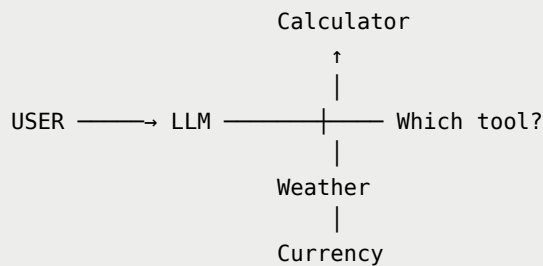
```

    """
public CurrencyResponse convertCurrency(
    double amount,
    String from,
    String to
) {

    return currencyService.convert(amount, from, to);
}

```

Now our LLM has three capabilities:



Our application can expose them together:

```

String response = chatClient
    .prompt()
    .user(message)
    .tools(
        calculatorTools,
        weatherTools,
        currencyTools
    )
    .call()
    .content();

```

From the user's perspective, we can still expose one endpoint:

POST /chat

We do not need separate AI-facing endpoints such as:

```
/calculator  
/weather  
/currency
```

The LLM determines which capability is needed from the user's natural-language request.

20. Multiple Tools for One Goal

Consider:

Convert \$120 into Indian rupees and then divide that amount equally among 6 people.

One tool call is not enough.

The process may become:

```
User  
↓  
LLM  
↓  
convertCurrency(  
    amount = 120,  
    from = USD,  
    to = INR  
)  
↓  
Currency Result  
₹X  
↓  
LLM  
↓  
calculate(  
    operation = divide,  
    a = X,  
    b = 6
```

```
)
↓
Calculator Result
₹Y
↓
LLM
↓
Final Answer
```

The important part is that we did not manually write:

```
currencyTool();
calculatorTool();
```

The model understands that the final goal requires multiple steps.

It decides:

```
First get the converted amount.

Then divide that result by six.
```

This begins to look much more **agentic**.

21. Another Multi-Tool Example

Consider:

I'm travelling to Delhi. What is the current weather there, and how much is \$500 in Indian rupees?

The model may require two independent tools:

```
LLM ————┐
           ├── getCurrentWeather("Delhi")
           └── convertCurrency(500, "USD", "INR")
```

One natural-language request can therefore involve multiple external systems while producing one coherent response.

22. What Actually Happens During Tool Calling?

Understanding the internal flow is important.

Step 1: User Sends a Message

```
What is 53 multiplied by 29?
```

The application sends the user request along with information about available tools.

Conceptually:

USER MESSAGE:

```
What is 53 multiplied by 29?
```

AVAILABLE TOOLS:

```
calculate(...)
getCurrentWeather(...)
convertCurrency(...)
```

Step 2: Model Requests a Tool

Instead of immediately generating the final answer, the model may respond conceptually with:

```
{
  "tool": "calculate",
  "arguments": {
    "operation": "multiply",
    "a": 53,
    "b": 29
  }
}
```

```
}  
}
```

This effectively means:

| Application, please execute this tool for me.

Step 3: Spring AI Executes the Tool

Spring AI identifies the requested Java method:

```
calculatorTools.calculate(  
    "multiply",  
    53,  
    29  
);
```

The JVM executes the Java code.

Result:

```
1537
```

An important distinction:

| **The LLM itself did not enter the JVM and execute Java code.**

The application executed Java **on behalf of the model**.

Step 4: Tool Result Goes Back to the LLM

Conceptually, the conversation now contains:

USER:

What is 53×29 ?

ASSISTANT:

```
Call calculate(  
    multiply,  
    53,  
    29  
)
```

TOOL:

1537

The LLM is invoked again with the tool result.

This means one API request from the user can internally involve multiple model interactions.

Step 5: Model Decides What to Do Next

The model now knows:

```
Tool result = 1537
```

It evaluates whether the user's goal has been completed.

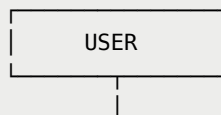
If yes, it generates the final response:

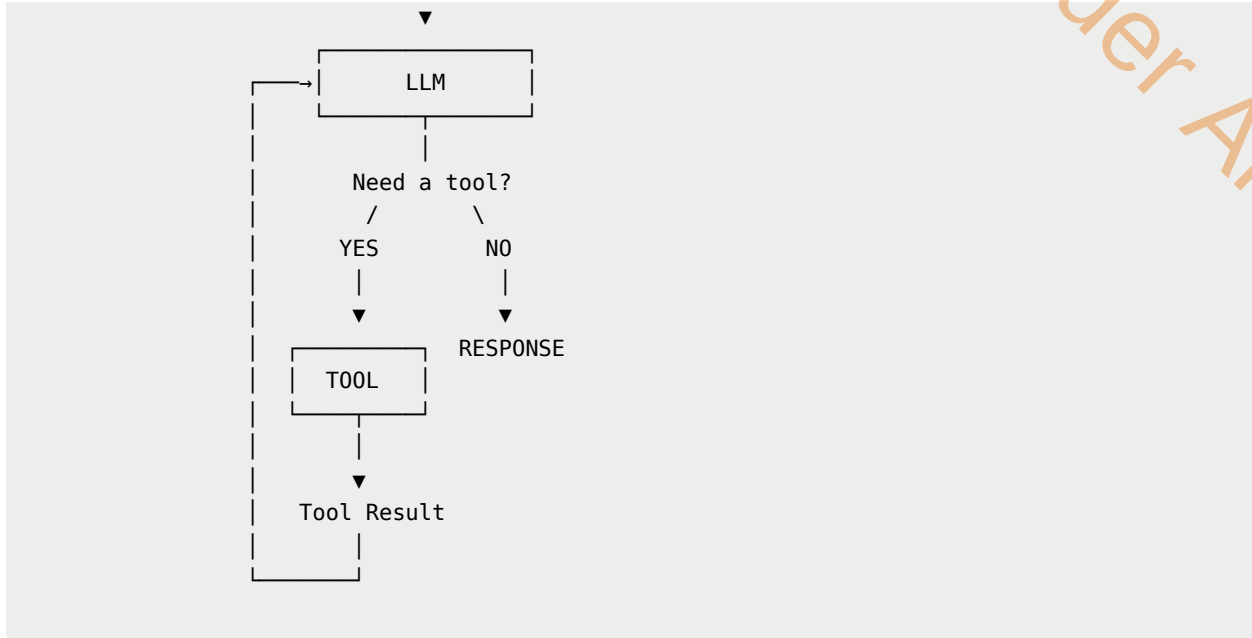
```
53 multiplied by 29 is 1,537.
```

If not, it may request another tool.

23. The Tool-Calling Loop

The overall architecture looks like this:





The most important part is the loop back to the LLM.

```

Think
↓
Act
↓
Observe
↓
Think Again
  
```

This loop becomes fundamental when building agents.

24. Two Broad Families of Tools

Tools can broadly be understood in two categories.

Information Tools

These retrieve information that the model does not currently have.

Examples:

- weather API
- currency rates

- databases
- search systems
- customer information
- inventory information

Their purpose is essentially:

Give me information I don't currently have.

Action Tools

These change something in the external environment.

Examples:

```
sendEmail()  
createUser()  
refundOrder()  
bookFlight()  
createFile()  
runCommand()
```

Their purpose is:

Do something for me.

The difference becomes very important when thinking about AI agents.

A weather tool lets the LLM **observe** the world.

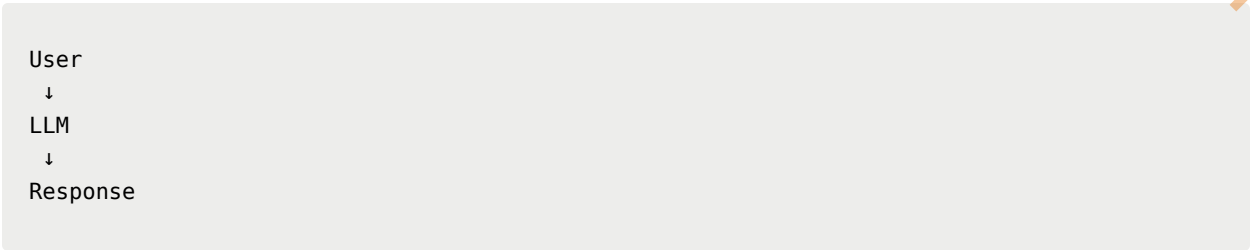
A file-writing tool lets it **change** the world.

25. When Does an LLM With Tools Become an Agent?

Simply giving an LLM a tool does not automatically give us the most useful definition of an agent.

It is better to think of agency as a progression.

Level 1: Raw LLM



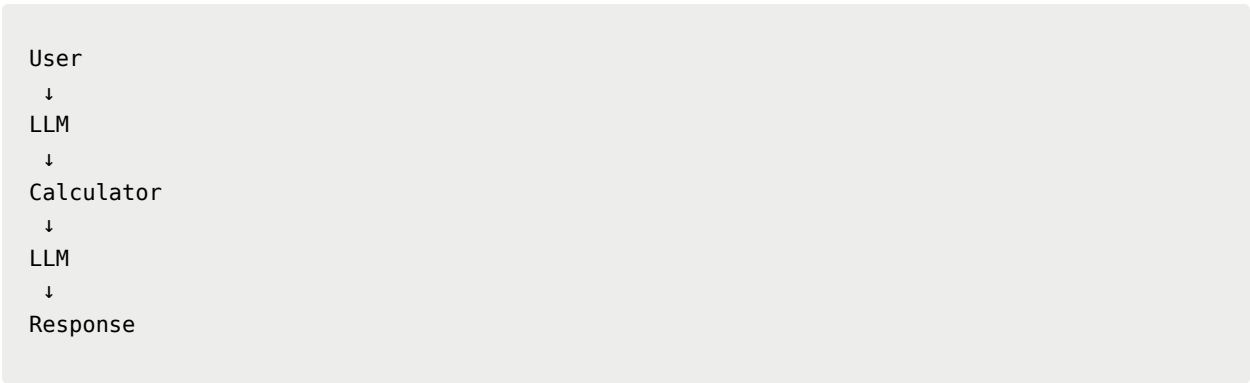
Example:

| Explain polymorphism.

The model generates text.

Nothing outside the model changes.

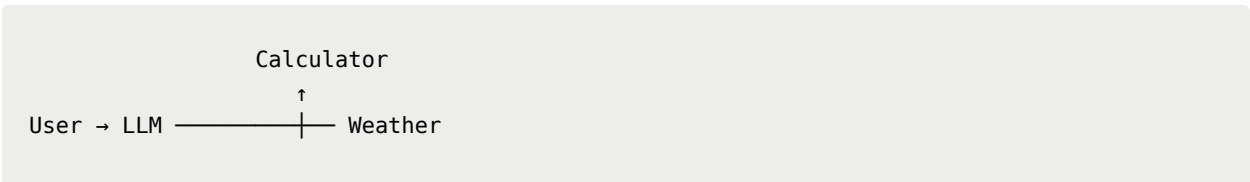
Level 2: LLM + One Tool



The model can decide to delegate a particular task.

This is **tool calling**.

Level 3: LLM + Multiple Tools



|
└─ Currency

Now the model must perform **tool selection**.

It has multiple possible actions and must choose the appropriate one.

Level 4: Multiple Steps Toward One Goal

Consider:

| Convert \$500 into INR and divide it equally between 7 people.

The model must:

```
Understand Goal
  ↓
Choose Currency Tool
  ↓
Observe Result
  ↓
Choose Calculator Tool
  ↓
Observe Result
  ↓
Generate Final Answer
```

Now the system is no longer performing only one isolated tool call.

It is making a sequence of decisions toward a goal.

This takes us much closer to an AI agent.

26. First-Principles Definition of an AI Agent

Instead of memorizing complicated definitions, an agent can be understood using six basic pieces.

1. Goal

Something needs to be achieved.

Example:

```
Build me a portfolio website.
```

2. Decision Maker

Something decides what to do next.

In our case:

```
LLM
```

Its recurring question is essentially:

```
What should I do next?
```

3. Actions

The agent needs capabilities.

These are its tools.

```
createDirectory()  
writeFile()  
readFile()  
listFiles()
```

4. Environment

The tools operate on something.

For our website-building agent:

```
Computer File System
```

5. Observations

After taking an action, the agent receives information about what happened.

Examples:

```
Directory created successfully.

File written successfully.

Current files:

index.html
style.css
script.js
```

These results allow the model to understand the current state of the environment.

6. Loop

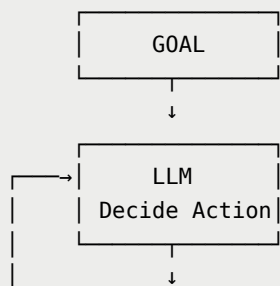
After receiving an observation, the agent evaluates:

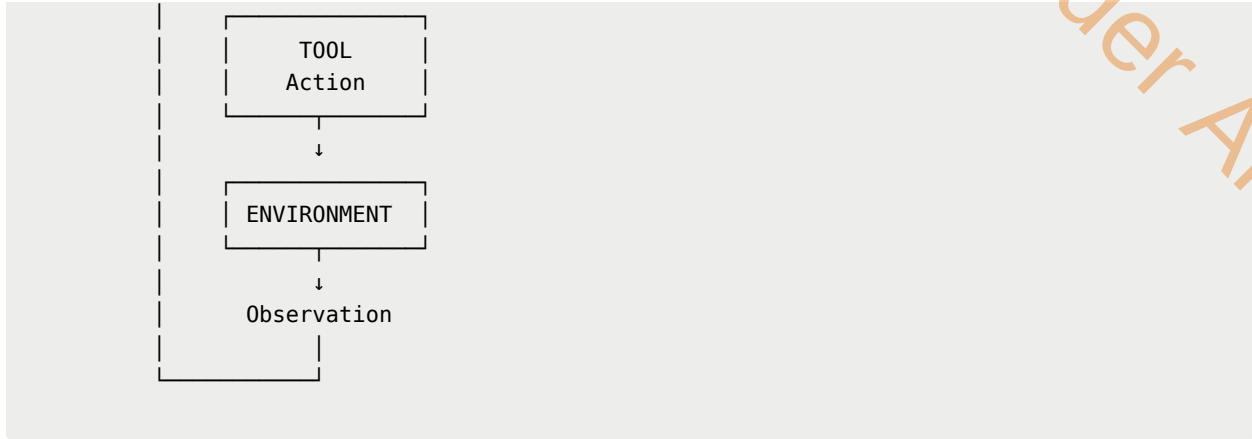
```
What should I do next?
```

It continues until:

```
Goal completed.
```

Putting everything together:





A useful mental model is:

Goal → Decide → Act → Observe → Decide Again

27. Agent vs Workflow

The distinction between a workflow and an agent can be simplified as:

WORKFLOW

We define:
WHAT + HOW

AGENT

We primarily define:
WHAT

The LLM decides much of:
HOW

For example, in a traditional workflow we may explicitly program:

Step 1 → Create directory
Step 2 → Create HTML
Step 3 → Create CSS
Step 4 → Create JavaScript

With an agent, we can instead provide:

```
Goal:
Build a website.

Available capabilities:
createDirectory
writeFile
readFile
listFiles
```

The agent decides how to use those capabilities.

Real-world systems often combine **workflows and agents** rather than using only one approach.

28. Practical Agent: AI Website Builder

Let's build an agent capable of creating websites.

The request can be extremely simple:

```
{
  "prompt": "Build a modern landing page for a premium coffee
shop called BrewLab."
}
```

After execution, we want:

```
generated-sites/
├─ brewlab/
│   ├─ index.html
│   ├─ style.css
│   └─ script.js
```

Opening `index.html` should give us an actual working website.

The important difference is that the model is no longer merely generating code in its response.

It is using tools to create files in an environment.

29. What Capability Is Missing?

An LLM already understands:

- HTML
- CSS
- JavaScript

So why can't it simply build the website?

Because:

Knowing how to write code is different from having permission and capability to write files.

Without tools:

```
"Create a website"
  ↓
  LLM
  ↓
"<html>...</html>"
```

It can tell us what the code should be.

What we want is:

```
"Create a website"
  ↓
  LLM
  ↓
Create index.html
  ↓
Create style.css
  ↓
Create script.js
```

The model needs hands.

30. Workspace Tools

For a simple website-building agent, four tools are enough:

```
createDirectory(path)

writeFile(path, content)

readFile(path)

listFiles(path)
```

With just these capabilities, the model can create surprisingly sophisticated static websites.

Notice what we are **not** giving it initially:

```
executeAnyTerminalCommand(String command)
```

That distinction is important for security.

31. Tool = Capability + Permission

We could expose something like:

```
@Tool
public String runCommand(String command) {
    ...
}
```

Then the model could potentially request:

```
runCommand("mkdir portfolio")

runCommand("touch portfolio/index.html")
```

But an unrestricted shell command is extremely powerful.

The process running the Spring application may have access to:

- files
- environment variables
- network resources
- application code
- system commands
- credentials

The more powerful the tool, the greater the risk.

A useful rule is:

| Tool = Capability + Permission

Giving an LLM access to a function is not only giving it a capability.

It is giving it permission to affect some part of the system.

32. Sandbox the Agent's Environment

Instead of allowing access to the entire machine, create a dedicated workspace.

```
project/  
├─ src/  
└─ generated-sites/
```

The agent should only be allowed to modify:

```
generated-sites/
```

It should not be allowed to access arbitrary locations such as:

```
/  
~/Documents
```

```
src/main/java
.env
```

This introduces an important production principle:

Sandbox the agent's environment.

The agent should receive only the minimum capabilities required to complete its job.

33. Building WorkspaceTools

Start with a controlled workspace:

```
@Component
public class WorkspaceTools {

    private final Path workspace =
        Path.of("generated-sites")
            .toAbsolutePath()
            .normalize();

}
```

Everything created by the agent will live inside this directory.

34. Tool 1: Create Directory

```
@Tool(description = """
    Creates a directory inside the website workspace.
    Use this when a new project or subdirectory is required.
    """)
public String createDirectory(
    @ToolParam(description = "Relative directory path")
    String path) {
```

```
try {
    Path target = safePath(path);
    Files.createDirectories(target);
    return "Directory created: " + path;
} catch (Exception e) {
    return "Failed to create directory: "
        + e.getMessage();
}
}
```

Now the model can affect the file system.

But importantly, it can only do so through a capability we explicitly provided.

35. Prevent Path Traversal

Suppose the model requests:

```
../../something
```

We do not want it escaping the allowed workspace.

A helper method can validate every path.

```
private Path safePath(String path) {
    Path resolved =
        workspace.resolve(path)
            .normalize();
}
```

```
if (!resolved.startsWith(workspace)) {  
    throw new IllegalArgumentException(  
        "Path outside workspace is not allowed"  
    );  
}  
  
return resolved;  
}
```

Now:

```
brewlab/index.html
```

can resolve inside:

```
/generated-sites/brewlab/index.html
```

while:

```
../../secret.txt
```

is rejected.

This demonstrates a very important principle:

```
LLM decides WHAT it wants to do.
```

```
APPLICATION decides WHAT it is allowed to do.
```

The LLM should never be considered the security boundary.

Your application remains responsible for enforcing permissions.

36. Tool 2: Write File

Coder Army

```

@Tool(description = """
    Creates or overwrites a text file inside the website
    workspace.
    Use this for HTML, CSS and JavaScript files.
    """)
public String writeFile(
    @ToolParam(description = "Relative file path")
    String path,

    @ToolParam(description = "Complete contents of the fi
le")
    String content) {

    try {

        Path target = safePath(path);

        Files.createDirectories(target.getParent());

        Files.writeString(
            target,
            content,
            StandardCharsets.UTF_8
        );

        return "Successfully wrote file: " + path;

    } catch (Exception e) {

        return "Failed to write file: "
            + e.getMessage();

    }

}

```

The abstraction is important.

The LLM does not care whether the application internally uses:

```
Files.writeString(...)
```

or stores the content in:

- a database
- cloud storage
- S3
- another remote service

The model simply understands:

```
writeFile(path, content)
```

Good tools expose capabilities at the level the model actually needs.

37. Tool 3: Read File

```
@Tool(description = """
    Reads an existing text file from the website workspace.
    Use this when you need to inspect a file that has already
    been created before deciding what to change.
    """)
public String readFile(String path) {

    try {

        return Files.readString(
            safePath(path)
        );
    }
```

```

    } catch (Exception e) {

        return "Unable to read file: "
            + e.getMessage();
    }
}

```

This capability makes the agent considerably more interesting.

It can now perform a loop such as:

```

ACT
↓
writeFile()

OBSERVE
↓
readFile()

DECIDE
↓
Something needs improvement.

ACT
↓
writeFile() again

```

The ability to observe the result of previous actions is a major part of agentic behaviour.

38. Tool 4: List Files

```

@Tool(description = """
    Lists files and directories inside a website project.
    Use it to inspect the current project structure.
    """)
public String listFiles(String path) {

```

```
...  
}
```

The model can call:

```
listFiles("brewlab")
```

and receive:

```
index.html  
style.css  
script.js
```

This gives the model another way to inspect the current environment.

39. Building the Agent's ChatClient

The tools can now be made available to the model.

```
ChatClient chatClient =  
    ChatClient.builder(chatModel)  
        .defaultTools(workspaceTools)  
        .build();
```

The tool-calling mechanism can repeatedly execute requested tools, return their results to the model and continue until the model reaches a final response.

This produces the loop:

```
LLM  
↓  
Tool Request  
↓  
Java Method  
↓  
Tool Result  
↓  
LLM
```

↓
Next Tool or Final Response

40. Give the Agent a Job Description

The system prompt defines the role and goal of the agent.

```
You are an expert frontend website builder.

Your job is to create complete static websites using the
available workspace tools.

For each website:

1. Create a dedicated project directory.
2. Create index.html.
3. Create style.css.
4. Create script.js when useful.
5. Build a polished responsive website.
6. Use semantic HTML.
7. Make the website visually impressive.
8. Do not use external build tools or frameworks.
9. Keep all files inside the provided workspace.
10. After creating the files, inspect the project structure.
11. Read important files back if needed and fix obvious issues.
12. Finish only when the website is complete.

Do not merely tell the user what code they should write.
Actually create the files using your tools.
```

The final instruction is particularly important:

Don't describe the action. Perform it.

Without a clear role, the model may simply generate HTML in the chat response.

We want the model to use the tools to change the environment.

41. The Controller Becomes Very Simple

```

@PostMapping("/generate-site")
public String generateSite(
    @RequestBody WebsiteRequest request) {

    return chatClient
        .prompt()
        .user(request.prompt())
        .call()
        .content();
}

```

From the controller's perspective, there is very little orchestration logic.

The model and tool-calling loop handle much of the decision-making.

42. Generate the First Website

Request:

```

{
  "prompt": "Create a premium landing page for a coffee shop
called BrewLab. Use a dark elegant design with a hero section,
featured coffees, customer testimonials and a contact section."
}

```

The agent may decide to create:

```

generated-sites/
├── brewlab/
│   ├── index.html
│   ├── style.css
│   └── script.js

```

The important point is that the files are actually created.

The model is no longer simply explaining how to build a website.
It is performing actions toward the goal.

43. General-Purpose Behaviour

Now send another request without changing the Java code:

```
{  
  "prompt": "Create a futuristic landing page for a company c  
alled NeuralForge that builds AI robots. Use a dark space-lik  
e design with glowing cards and subtle animations."  
}
```

The same agent may create:

```
neuralforge/  
├─ index.html  
├─ style.css  
└─ script.js
```

We did not build:

```
Coffee Shop Generator
```

We built:

```
Website-Building Agent
```

The user's goal changes.

The available tools stay the same.

The model determines how those capabilities should be used to accomplish each goal.

44. Human-in-the-Loop

Not every tool should necessarily execute immediately.

Suppose an agent has:

```
sendEmail()
```

Automatically executing ordinary emails may be acceptable in some systems.

But consider:

```
refundCustomer(₹50,000)
```

Immediately executing every model request would be dangerous.

Instead of:

```
LLM requests refund
    ↓
Execute immediately
```

we may introduce approval:

```
LLM requests refund
    ↓
  PAUSE
    ↓
Human approves?
  /      \
YES       NO
  ↓       ↓
Execute  Reject
```

This is known as **human-in-the-loop**.

It becomes especially important when tools can perform:

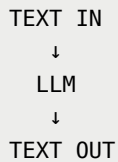
- financial transactions
- destructive operations

- external communication
- security-sensitive operations
- irreversible actions

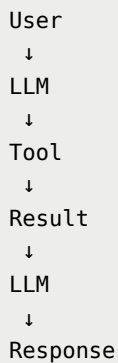
The more consequential the action, the stronger the controls around the tool should be.

45. The Complete Mental Model

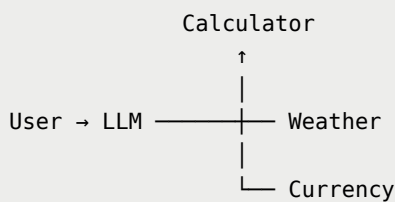
We began with a raw LLM:



Then we added tools:



Then multiple tools:



Finally, we introduced an environment and a loop:

```
GOAL
↓
LLM decides
↓
TOOL acts
↓
ENVIRONMENT changes
↓
OBSERVATION
↓
LLM decides again
↓
...
↓
GOAL COMPLETED
```

That is the foundation of an AI agent.

Key Takeaways

- An LLM fundamentally generates tokens; it does not automatically perform actions in the outside world.
- **LLM = Brain, Tools = Hands.**
- Tools are ordinary application functions exposed to the model through a structured contract.
- The LLM can translate natural-language intent into structured tool calls.
- The application—not the LLM—actually executes Java methods, APIs, database operations or file-system actions.
- Tool results are returned to the model so it can decide what to do next.
- One user request can involve multiple tool calls and multiple LLM interactions.
- Information tools retrieve or compute information.
- Action tools change the external environment.

- An agent combines a **goal, decision maker, tools, environment, observations and a loop.**
- In a workflow, we usually define **what and how**. In an agent, we primarily define **what**, while the model decides much of the **how**.
- Tools should expose only the capabilities the agent actually requires.
- **Tool = Capability + Permission.**
- The application must enforce security boundaries.
- Agents that interact with file systems or other environments should be sandboxed.
- High-impact actions may require **human-in-the-loop approval**.

Final Formula

```
LLM
+
Tools
+
Environment
+
Observations
+
Decision Loop
+
Goal
=
AI Agent
```

The important shift is simple:

A chatbot primarily answers. An agent can decide, act, observe the result, and continue working toward a goal.